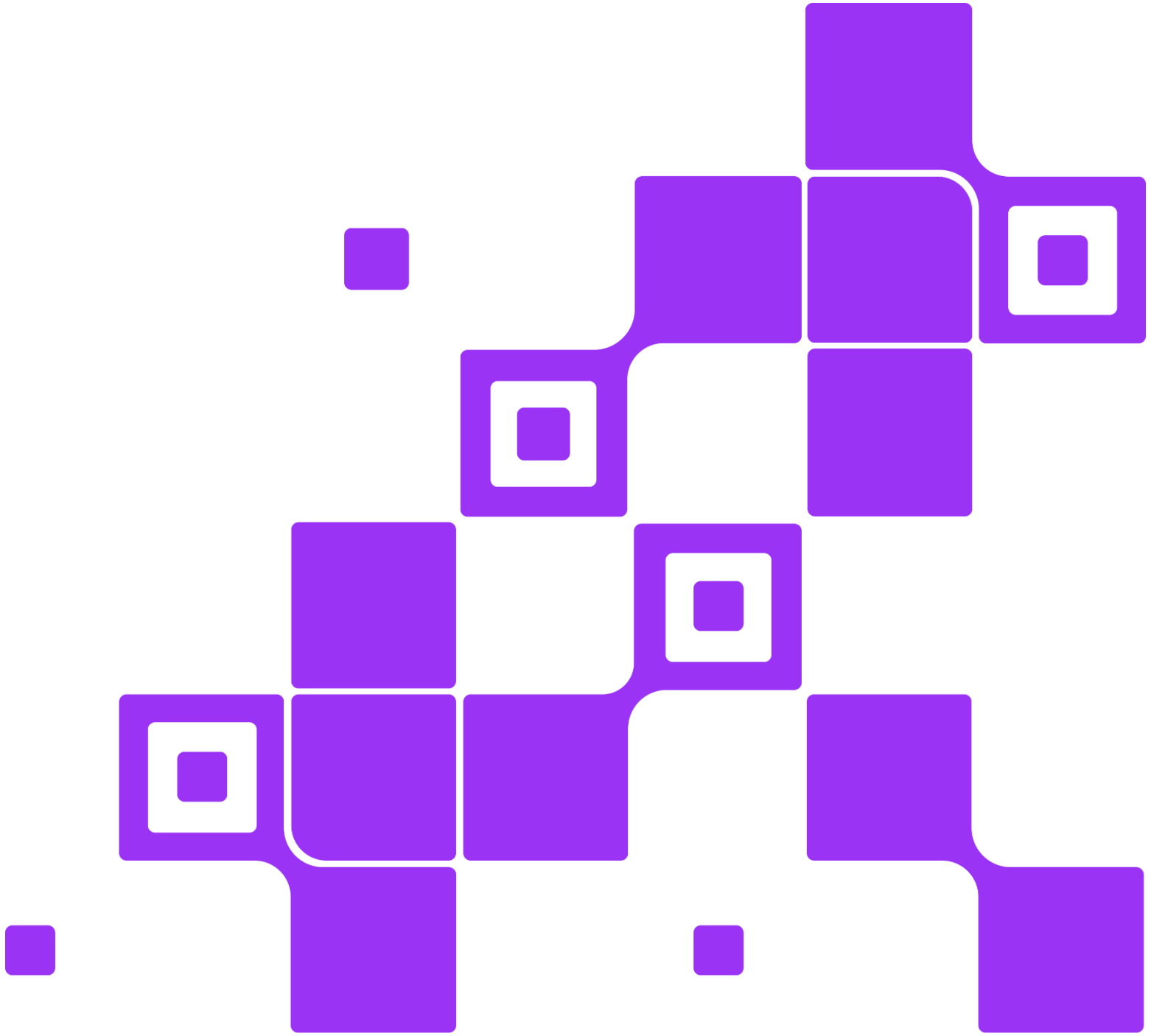
Класс Object



Класс Object

```
public class Object {
    ...
}
```

- Object — **самый фундаментальный класс** Java
- от Object наследуются практически все классы Java
- Object находится в пакете java.lang
- его не нужно импортировать вручную
- у любого обычного Java-класса есть методы Object

```
class User {

}
```

=

```
class User extends Object {

}
```

Почему Object вообще нужен?

IThub

Главная идея — полиморфизм и единый базовый тип

```
Object a = "Hello";
Object b = 42;
Object c = new User();
Object d = new int[] {1, 2, 3};
```

```
public void print(Object obj) {
    System.out.println(obj);
}
```

```
print("Hello");
print(123);
print(new User());
print(new ArrayList<>());
```

Методы класса Object

IThub

- public final native Class<?> **getClass**();
- public native int **hashCode**();
- public boolean **equals**(Object obj);
- protected native Object **clone**() throws CloneNotSupportedException;
- public String **toString**();
- public final void **notify**();
- public final void **notifyAll**();
- public final void **wait**() throws InterruptedException;
- public final void **wait**(long timeout) throws InterruptedException;
- public final void **wait**(long timeout, int nanos) throws InterruptedException;

Метод toString()

IThub

public String toString()

возвращает строковое представление объекта

```
class User {
    String name;
    int age;
}
```

Стандартная реализация Object.toString() строит строку примерно из: имя класса + @ + hash-код в шестнадцатеричном виде

```
User user = new User();

user.name = "Alex";
user.age = 25;

System.out.println(user);
```

```
User@5acf9800
```

Метод toString()

IThub

```
class User {

    String name;
    int age;

    @Override
    public String toString() {
        return "User{name='" + name + "', age=" + age + "}";
    }
}
```

```
User user = new User();

user.name = "Alex";
user.age = 25;

System.out.println(user);
```

```
User{name='Alex', age=25}
```

Переопределять метод toString() очень полезно для:

- логирования
- отладки (Debugging)
- вывода объектов
- работы с коллекциями

Метод equals()

IThub

public boolean equals(Object obj)

отвечает за определение: являются ли два объекта логически равными?

Вот здесь начинаются важные нюансы.

== и **equals()** — не одно и то же!

```
String a = new String("hello");
String b = new String("hello");
```

Объекты разные:
a == b вернет false
Потому что **==** для объектов **сравнивает ссылки.**

a.equals(b) вернет true
Потому что String переопределяет equals() и **сравнивает содержимое строк**.

Как работает equals() из Object?

IThub

```
class User {
    String name;
}
```

```
User a = new User();
User b = new User();

a.name = "Alex";
b.name = "Alex";

System.out.println(a.equals(b));
```

```
false
```

Потому что стандартный Object.equals() фактически сравнивает: this == obj
Поэтому два разных экземпляра не равны, даже если внутри у них одинаковые поля.

Метод equals()

```
class User {
    String name;
    int age;

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }

        if (obj == null) {
            return false;
        }

        if (getClass() != obj.getClass()) {
            return false;
        }

        User other = (User) obj;
        return age == other.age && Objects.equals(name, other.name);
    }
}
```

Если мы хотим, чтобы пользователи с одинаковыми данными считались одинаковыми

Контракт equals()

IThub

У equals() есть важный контракт.

Метод должен быть:

- **Reflexive -** объект равен самому себе: a.equals(a) == true
- **Symmetric** - если: a.equals(b) то должно быть: b.equals(a)
- **Transitive** - если: a == bb == c то a == c
- **Consistent** - если состояние объектов не изменилось, повторный вызов должен давать тот же результат.
- **Null** - для ненулевого объекта a : a.equals(null) должно возвращать false

Метод hashCode()

IThub

public int hashCode() возвращает числовой hash объекта.

```
User user = new User();

System.out.println(user.hashCode());
```

```
123456789
```

Но самое важное — не само число, а **контракт между equals() и hashCode()**.

Контракт между equals() и hashCode()

IThub

Равные объекты обязаны иметь одинаковый hash code.

Если a.equals(b) == true то обязательно a.hashCode() == b.hashCode()

Но обратное неверно

Может быть a.hashCode() == b.hashCode()

Но a.equals(b) == false

Это называется коллизией

Пример коллизии

IThub

В Java у String хеш считается так: $hash = s[0] * 31^{n-1} + s[1] * 31^{n-2} + \cdots + s[n-1]$

Для строки "Aa“ (n=2):
'A' = 65
'a' = 97

$$hash = s[0] * 31^{2-1} + s[1]$$
$$= 65 * 31 + 97 = 2015 + 97 = 2112$$

Для строки "BB":
'B' = 66
'B' = 66

$$hash = s[0] * 31^{2-1} + s[1]$$
$$= 66 * 31 + 66 = 2046 + 66 = 2112$$

```
public static void main(String[] args) {

    String a = "Aa";
    String b = "BB";

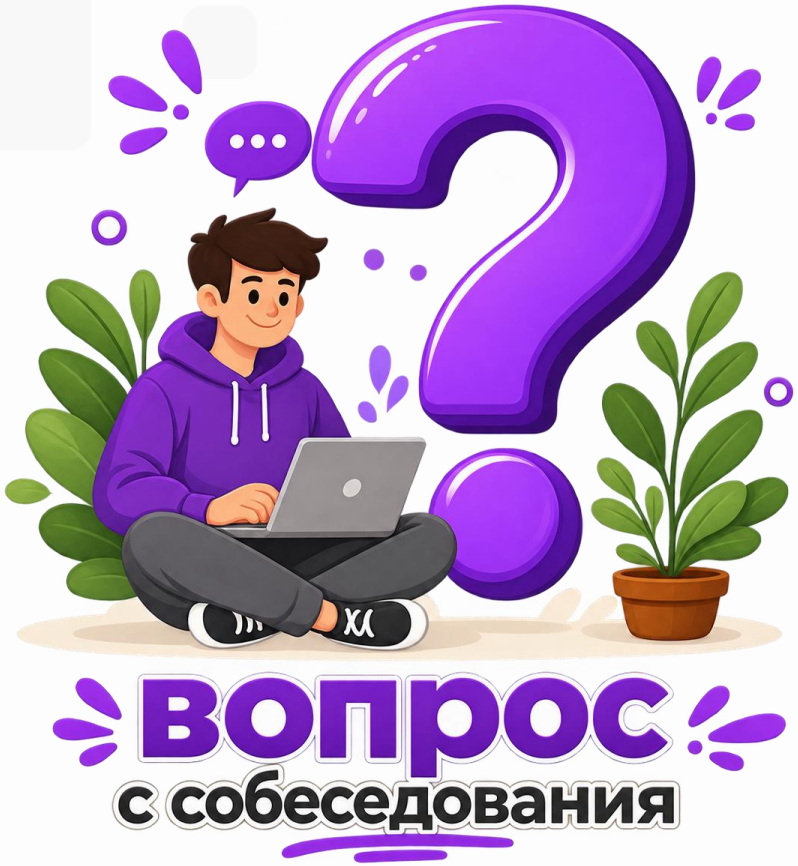
    System.out.println(a.hashCode());
    System.out.println(b.hashCode());

    System.out.println(a.equals(b));
  }
```

```
2112
2112
false

Process finished with exit code 0
```

IThub

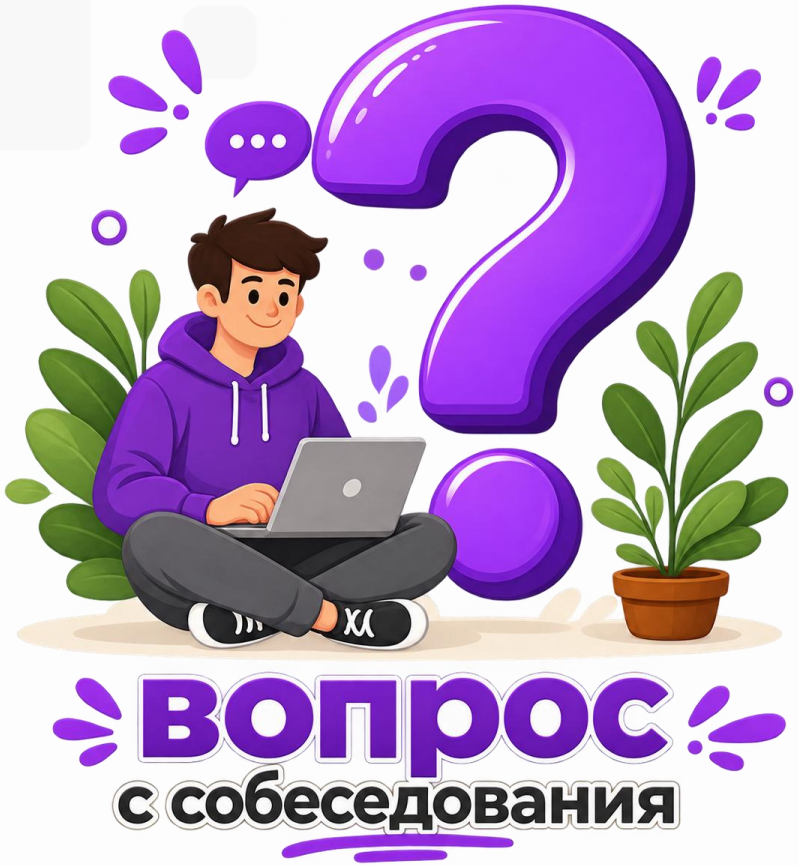


Зачем нужен equals(). Чем он отличается от операции ==?

Метод equals() - определяет отношение эквивалентности объектов.

При сравнении объектов с помощью == сравнение происходит лишь между ссылками. При сравнении по переопределённому разработчиком equals() - по внутреннему состоянию объектов.

IThub



Какие поля следует использовать при подсчете hashCode()?

Лучше выбирать поля, которые наверняка будут различаться.

Для этого необходимо использовать уникальные, лучше всего примитивные поля, например, такие как id, uuid.

При этом нужно следовать правилу, если поля задействованы при вычислении hashCode(), то они должны быть задействованы и при выполнении equals().

Почему hashCode() настолько важен?

IThub

Из-за коллекций: HashMap HashSet HashTable

Set<User> users = new HashSet<>();

Когда мы добавляем в коллекцию новый объект **users.add(user)**

HashSet использует hashCode() для определения, куда поместить объект, а затем equals() для проверки равенства кандидатов

```
class User {
    String name;
    int age;

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }

        if (obj == null) {
            return false;
        }

        if (getClass() != obj.getClass()) {
            return false;
        }

        User other = (User) obj;
        return age == other.age && Objects.equals(name, other.name);
    }
}
```

Мы переопределили equals(), но забыли hashCode().

```
User a = new User("Alex");
User b = new User("Alex");

System.out.println(a.equals(b));
```

```
true
```

```
Set<User> users = new HashSet<>();
users.add(a);
users.add(b);
```

может содержать два объекта, хотя a.equals(b) == true

```
import java.util.Objects;

class User {

    private String name;
    private int age;

    @Override
    public boolean equals(Object obj) {

        if (this == obj) {
            return true;
        }

        if (!(obj instanceof User other)) {
            return false;
        }

        return age == other.age && Objects.equals(name,
other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }
}
```

IThub

Как правильно написать equals() + hashCode()

Метод getClass()

IThub

public final Class<?> getClass()

возвращает объект Class, который описывает runtime-класс объекта.

```
User user = new User();

System.out.println(user.getClass());
```

```
class User
```

```
System.out.println(user.getClass().getName());
```

```
com.example.User
```

```
System.out.println(user.getClass().getSimpleName());
```

```
User
```

Метод getClass() и наследование

IThub

getClass() показывает реальный класс объекта во время выполнения, а не тип ссылки:

```
class Animal {
}

class Dog extends Animal {
}
...
Animal animal = new Dog();
```

Тип переменной Animal, но реальный объект Dog

```
animal.getClass()
```

```
Dog.class
```

Метод clone()

IThub

В Object существует метод для создания копии объекта

protected native Object clone() throws CloneNotSupportedException;

```
class User implements Cloneable {

    String name;

    @Override
    public User clone() throws CloneNotSupportedException {
        return (User) super.clone();
    }
}
```

```
User a = new User();
a.name = "Alex";

User b = a.clone();
```

a == b вернет false

Object.clone() делает поверхностное копирование объекта — **shallow copy**.

Для глубокого копирования (deep copy) нужно переопределять clone() и клонировать вложенные объекты вручную.

Лучше использовать явный метод копирования. Например:

```
class User {

    String name;
    int age;

    User(User other) {
        this.name = other.name;
        this.age = other.age;
    }
}
```

Использование:

```
User copy = new User(original);
```

Задание

```
public class User {
    private String name;
    private int age;

    public User(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // TODO: реализовать методы Object - toString() equals() hashCode()
}
```

Два пользователя являются одинаковыми, если у них совпадают name и age